

**COHERENT TEMPORAL SYNTHESIS FOR INCREMENTAL
ACTION SEGMENTATION**

by

HANS CHRISTIAN CHUA GOLONG

(B.S., Ateneo De Manila University)

A THESIS SUBMITTED FOR THE DEGREE OF

MASTER OF COMPUTING

in

ARTIFICIAL INTELLIGENCE

in the

SCHOOL OF COMPUTING

of the

NATIONAL UNIVERSITY OF SINGAPORE

2024

Supervisor:

Associate Professor Angela Yingjie Yao

Examiners:

Associate Professor Angela Yingjie Yao

Assistant Professor Mike Zheng Shou

Declaration

I hereby declare that this thesis is my original work and it has been written by me in its entirety. I have duly acknowledged all the sources of information which have been used in the thesis.

This thesis has also not been submitted for any degree in any university previously.

Hans Christian Chua Golong

30 April 2024

Acknowledgments

I would like to express my deepest gratitude to Prof. Angela Yao for taking a chance with me and letting me work in her lab. She was one of my first teachers in computer vision and has since been a mentor to me throughout my stay in NUS. I have learned so much over the past year, and I am greatly indebted to her for helping me find my passion in computer vision.

I would like to thank Dr. Guodong Ding for his unwavering support and endless patience. Working with him closely over this project, he showed me the ropes on how to do good research and has been an invaluable source of encouragement and advice. He is a role model that I look up to and the kind of researcher that I aspire myself to be.

I would like to thank the people of the CVML lab for being so approachable and welcoming. The warm environment fostered in the lab is truly unlike any other I have seen, and I hope I get another chance to work in the group again.

Finally, I would like to thank my mother for her unconditional love and constant encouragement. I would not be who I am today without her.

Contents

Acknowledgments	i
Abstract	v
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background	1
1.2 Incremental Temporal Action Segmentation (iTAS)	3
1.3 Research Challenges	4
1.3.1 Incremental Learning in the Video Domain	4
1.3.2 Temporally Coherent Video Generation	4
1.4 Contributions and Publication	5
1.5 Outline	5
2 Literature Review	7
2.1 Temporal Action Segmentation (TAS)	7
2.1.1 Video Features	7
2.1.2 Segmentation Models	8
2.2 Class Incremental Learning (CIL)	12
2.2.1 Common Approaches	13
2.2.2 Data Replay in the Video Domain	14
3 Proposed Methodology	16
3.1 Problem Formulation	16
3.1.1 Temporal Action Segmentation	16

3.1.2	Class Incremental Learning	17
3.1.3	Incremental Temporal Action Segmentation	18
3.2	Generative Data Replay with a Temporally Coherent Action Model	18
3.2.1	Temporally Coherent Action (TCA) Model	18
3.2.2	Replay Data Generation	20
3.2.3	iTAS Training	21
4	Experiments	23
4.1	Datasets	23
4.1.1	Breakfast	23
4.1.2	CrossTask	24
4.1.3	YouTube Instructional	24
4.1.4	Adapting for iTAS	25
4.2	Evaluation Metrics	25
4.2.1	Frame-wise Metrics	25
4.2.2	Segment-wise Metrics	26
4.2.3	Evaluation in iTAS	26
4.3	Implementation Details	27
4.3.1	TCA Model	27
4.3.2	TAS Backbones	27
4.3.3	Replay Size	27
4.4	Effectiveness	28
4.4.1	Baselines	28
4.4.2	Results	28
4.5	Ablation Studies	30
4.5.1	Feature Diversity and Temporal Coherence	30
4.5.2	Replay Size	31
4.5.3	TCA Training Data	31
4.5.4	Task Sequence	32
4.5.5	Visualizations	33
5	Conclusion	36
5.1	Limitations	36

5.2 Future Work	36
Bibliography	38

Abstract

Coherent Temporal Synthesis for Incremental Action Segmentation

by

Hans Christian Chua Golong

Master of Computing in Artificial Intelligence

National University of Singapore

Data replay is a successful incremental learning technique for images. It prevents catastrophic forgetting by keeping a reservoir of previous data, original or synthesized, to ensure the model retains past knowledge while adapting to novel concepts. However, its application in the video domain is rudimentary, as it simply stores frame exemplars for action recognition. This thesis presents the first exploration of video data replay techniques for incremental action segmentation, with a focus on action temporal modeling. We propose a Temporally Coherent Action (TCA) model, which represents actions using a generative model instead of storing individual frames. The integration of a conditioning variable that captures temporal coherence allows our model to understand the evolution of action features over time. Therefore, action segments generated by TCA for replay are diverse and temporally coherent. In a 10-task incremental setup on the Breakfast dataset, our approach achieves significant increases in accuracy for up to 22% compared to the baselines.

List of Figures

1.1	Overview of Temporal Action Segmentation. From [9].	2
1.2	Overview of Incremental Learning	2
2.1	Overview of TCNs Proposed by Lea et al. From [32].	9
2.2	Overview of MSTCN. From [11].	10
2.3	Overview of ASFormer. From [74].	11
3.1	Temporally Coherent Action (TCA) model.	19
3.2	Replay Data Generation.	21
3.3	Learning schemes for temporal action segmentation.	22
4.1	Performance following each incremental step with the varied task sequence.	32
4.2	Comparsion of confusion matrices of different approaches on Breakfast in a 10-task setup.	33
4.3	T-SNE visualizations of generated action segments.	34
5.1	Effect of task sequence order to training set composition and model performance.	37

List of Tables

4.1	Summary of iTAS Datasets.	23
4.2	Performance comparison on Breakfast, CrossTask, and YouTube Instructional.	28
4.3	Performance comparison on Breakfast with the blurry task boundary. .	29
4.4	Ablation study on the components in TCA.	30
4.5	Different replay sizes on Breakfast.	31
4.6	Different ratios for TCA training on Breakfast.	31

Chapter 1

Introduction

Video understanding is one of the hallmark tasks in computer vision. Designing systems that can infer information from videos have exciting applications in surveillance, quality assurance, robotics, virtual assistants, and many other areas.

Action recognition [27] is one of the foremost tasks in video understanding. Action recognition systems aim to take a video and identify the action demonstrated. Many action recognition methods, however, require the videos to be short pre-trimmed clips, while real-world videos are usually minutes long or come in the form of streams.

Hoping to overcome this limitation, temporal action segmentation aims to take an untrimmed video and divide it into a sequence of action segments, each of which is then associated with an action label [9]. The approaches used in the field have benefited significantly from the success of deep learning from image, language, and speech domains. However, even with recent developments, we discover that incremental learning, a learning paradigm where a model is incrementally updated with new information, has not yet been explored in the field.

In this work, we define an incremental learning setup for temporal action segmentation and propose a novel generative data replay approach for incremental procedural video understanding.

1.1 Background

Temporal action segmentation (TAS) is a challenging task of video understanding that deals with untrimmed goal-oriented procedural videos. The objective in TAS is two-fold. It aims to separate the procedural video into contiguous action segments. Then, it aims to assign an action label to each segment (see Fig. 1.1). In the parlance of TAS, a procedural video demonstrates a *complex activity* (e.g. “coffee”), which is

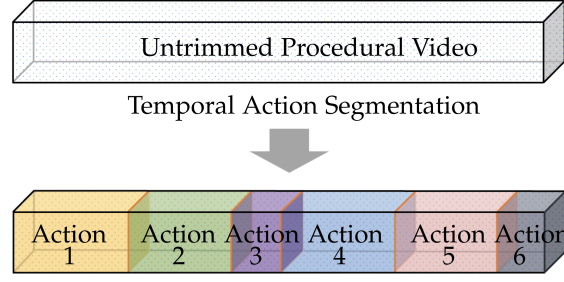


Figure 1.1: Overview of Temporal Action Segmentation. From [9].

composed of a sequence of *actions* (e.g. “*pour coffee*”, “*pour milk*”) [9]. Although videos belonging the same complex activity have a similar goal to achieve, the individual actions taken to accomplish the goal do not necessarily have to the same. Actions may be done in a different sequence order, and some actions may be skipped entirely.

While several methods have been proposed for TAS, the models developed are static and need to be retrained completely once new information is available. Retraining from scratch incurs significant time and computational costs. Past data may not always be available for retraining as well, due to physical storage limitations or privacy concerns [8]. TAS algorithms need to be designed with the ability to update models with new data.

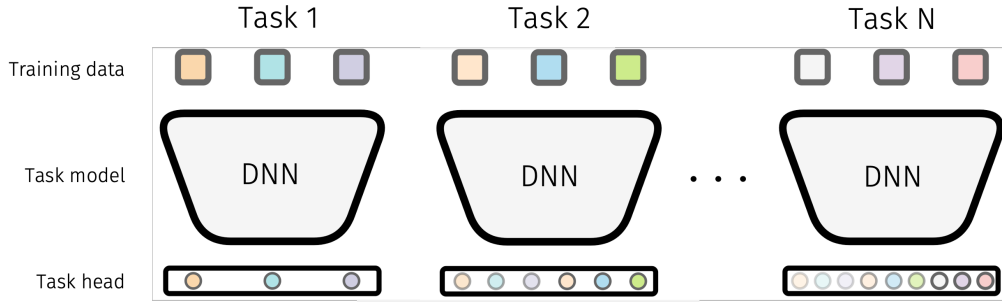


Figure 1.2: Overview of Incremental Learning

Incremental learning (IL) [8] is the field dedicated to giving models the capability to learn on new data without forgetting previous knowledge. In IL, datasets are divided into *tasks*, and the model is incrementally trained on one task at a time without access to data from other tasks (see Fig. 1.2). Naively training models in this manner causes a phenomenon called catastrophic forgetting [13], where a

model will perform poorly on past tasks after being trained on a new task. Thus, IL approaches are developed to mitigating the catastrophic forgetting problem.

1.2 Incremental Temporal Action Segmentation (iTAS)

Just as humans constantly learn from everyday experiences, we need to design models with the ability to incrementally learn from new data. Towards this goal, we introduce the incremental temporal action segmentation (iTAS) task. In iTAS, the dataset is split into activities (e.g. “*coffee*”), where each activity is composed of a set of actions (e.g. “*pour coffee*”, “*pour milk*”). At each incremental step, the model is trained on a different activity and has no access to the data of prior or future activities. The TAS model is forced to learn new action classes while retaining previous knowledge.

While incremental learning in computer vision was initially restricted to images, it has recently been explored in the video domain with incremental action recognition [68, 3, 46, 47], where many approaches use data replay. Data replay [50] is an intuitive IL approach that prevents catastrophic forgetting by providing the model with samples of prior tasks for rehearsal. The samples, also called exemplars, of prior tasks are stored in a replay buffer with a fixed memory size.

However, since existing action recognition models [71, 41] accept sparse frames as input, keeping an exemplar set for each action clip does not overly strain the memory allocation. Additionally, because of a strong static bias towards scene context [38] in standard action recognition datasets [7, 63], comparable performance can be achieved by storing entire segments, a few frames [68], or even a single frame [3] for data replay, all without needing explicit temporal modeling.

We advocate exploring data replay strategies for incremental video understanding with iTAS. The static bias is less pronounced in TAS since actions in a sequence are likely to share a common background, and videos in TAS also need to retain a high temporal resolution to allow for accurate frame-wise predictions.

1.3 Research Challenges

1.3.1 Incremental Learning in the Video Domain

Following prior works, our approach uses data replay for incremental video understanding. Compared to images, videos require significantly more memory to store, making adapting image-based approaches directly to videos a challenge.

The difficulty is emphasized in iTAS. TAS models operate on full-resolution frame-by-frame video inputs and produce action predictions at the same level of temporal granularity. Naively storing frame exemplars in proportion to the video’s duration, even downsampled, would impose a significant memory burden. Using sub-sampling to store sparse frames as exemplars per video would result in a loss of temporal coherence in the natural progression of actions, which would harm the fine-grained frame-wise predictions.

To address the problem of needing full-resolution replay data without incurring significant spatial cost, we elect to use a generative model to create samples for data replay. Our design choice is motivated by the the model’s capacity to learn efficient data representations with a fixed model size while producing diverse outputs of arbitrary lengths.

1.3.2 Temporally Coherent Video Generation

While generative data replay appears to be a straightforward solution for iTAS, video generation is in itself an open problem. For the generated videos to be useful for training, two key conditions need to be met; each synthesized frame needs to have plausibly come from the real-world distribution, and adjacent frames need to be temporally coherent.

In our approach, we tackle this challenge using a conditional variational auto-encoder (cVAE) [62]. We introduce a temporal coherence variable, which provides the relative temporal position of the frame to generate, and integrate it into the cVAE model to generate temporally coherent video samples. We empirically show that our approach achieves superior results compared to baselines. We also conduct ablation studies to verify the effectiveness of the temporal coherence variable on

maintaining temporal coherence and demonstrate the importance of using temporally coherent samples for training.

1.4 Contributions and Publication

Our work presents the following contributions:

1. To the best of our knowledge, we are the first to introduce the incremental action segmentation task, working with procedural videos. This is a natural fit for the development of intelligent assistants which learn complex tasks and activities in the real world in an incremental manner.
2. We propose to model actions with generative models and use the models to generate diverse replay data. The proposed generative data replay bypasses the trade-off problem in frame-exemplar storing approaches.
3. We introduce a temporal coherence variable to help the generative model learn action feature evolution and produce temporally coherent action segments in replay video generation.
4. Experiments on three procedural benchmarks show that our approach can effectively mitigate catastrophic forgetting for incrementally learned action segmentation models.

This thesis is based on a joint work with Dr. Guodong Ding. The paper from this project has been accepted into CVPR 2024:

- Guodong Ding, **Hans Golong**, and Angela Yao, “Coherent Temporal Synthesis for Incremental Action Segmentation”, in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

1.5 Outline

This thesis will be organized as follows. In Chapter 2, we conduct a review of existing literature. We will discuss previous work in temporal action segmentation

CHAPTER 1. INTRODUCTION

and incremental learning. In Chapter 3, we will discuss our generative data replay approach and detail our proposed Temporally Coherent Action (TCA) model. In Chapter 4, we will present our experimental results and discuss the findings from our ablation studies. Finally, we close in Chapter 5 with a summary of our work and discuss potential directions for future research.

Chapter 2

Literature Review

2.1 Temporal Action Segmentation (TAS)

Generally, approaches in temporal action segmentation can be decoupled into two steps. First, appearance and short-term motion information from the input video is encoded into per-frame features. Then, dedicated segmentation models, designed to learn long-range dependencies from the features, are used to predict per-frame action labels.

TAS approaches typically do not train these two tasks in an end-to-end model, due to the complexity of learning video features. For work focused on the design of segmentation models, it has become standard to use pre-computed frame-wise features. Through this practice, models can be compared fairly without the effect of variances in the feature extraction phase.

2.1.1 Video Features

Video feature extraction methods are traditionally based on approaches from activity recognition. While representation learning in TAS is an area of active research [36, 21], two older feature representations that are still widely used are Fisher Encoded Improved Dense Trajectories (IDT) and Inflated 3D ConvNets (I3D).

The original work on dense trajectories (DT) [69] designed a hand-crafted trajectory-aligned feature representation. Classical feature descriptors are extracted along the dense trajectories and encoded into a bag-of-features representation, which could then be used to perform action recognition. A mechanism to improve the calculated trajectories by suppressing the effect of camera motion was proposed in [70] to form improved dense trajectories (IDT). In the same work, Wang and Schmid

reported that encoding the trajectory-aligned descriptors into Fisher vectors [48] improved performance compared to the previous bag-of-features approach. Fisher encoded IDT features are now widely used in many standard TAS datasets.

I3D [7] is a two-stream deep network feature extractor based on an inflated Inception-V1 [19] image classification network. The Inception-VI network is first pre-trained on ImageNet [56], then the $N \times N$ filters of the network are expanded temporally to $N \times N \times N$. The inflated network is bootstrapped with the pre-trained weights by replicating the parameter values into the new temporal dimension and scaling them by $\frac{1}{N}$. The I3D network is pre-trained on the the Kinetics Human Action Video Dataset [23] before being used to extract features. The I3D network has an RGB stream and an optical flow stream, each of which outputs a 1024-D feature vector. The 1024-D vectors can either be used on their own or be concatenated to a 2048-D representation.

2.1.2 Segmentation Models

While the extracted video features can be used to make direct frame-wise action classifications, using them together with segmentation models specifically designed to capture long-range temporal relations can achieve better performance. Thus, much research has been dedicated to developing segmentation models.

2.1.2.1 Traditional Approaches

Traditional approaches that model long-range temporal relations include sliding windows [55, 45], language and probabilistic models [51, 29], and Recurrent Neural Networks (RNN) [52, 60]. However, these methods were still constrained to information in a local temporal neighborhood, an immediately prior state, or a fixed-size context vector, all of which may be insufficient to capture very long-term dependencies. Lea et al. identified this shortcoming and proposed temporal convolutional networks, which have since dominated the field.

2.1.2.2 Temporal Convolutional Networks

Temporal convolutional networks (TCN) [32] are a class of network architectures whose distinguishing design is a hierarchy of temporal convolutional layers that allow it to capture much more of an input’s temporal dependencies compared to previous

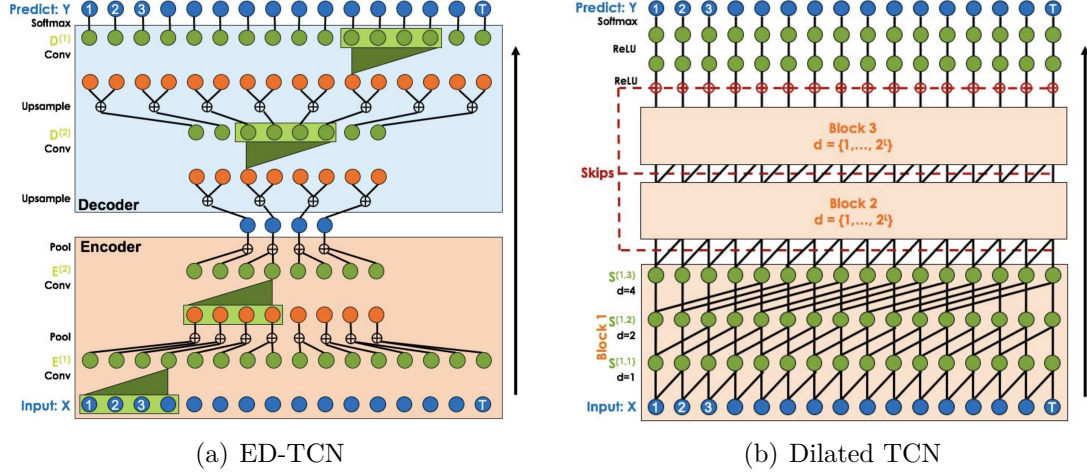


Figure 2.1: Overview of TCNs Proposed by Lea et al. From [32].

approaches. The first two TCN variants introduced by Lea et al. in [32] were the Encoder-Decoder TCN (ED-TCN) and the Dilated TCN.

ED-TCN (see Fig. 2.1(a)) takes inspiration from the encoder-decoder architecture. It uses an encoder to compress the input into a latent representation which is passed to the decoder to output frame-wise predictions. The encoder uses several temporal convolution and max-pooling operations to force the input features into a compact latent encoding. The temporal receptive field of the convolution operation grows at deeper levels in the encoder as the intermediate output is temporally shrunk with max-pooling. The decoder block then takes the encoder's condensed output and processes it through several upsampling and convolution operations to output the frame-wise action predictions. The temporal information in the condensed latent representation is distributed to the predictions as it moves through the decoder's layers.

Dilated TCN (see Fig. 2.1(b)) is an alternate architecture proposed that uses a sequence of dilated temporal convolution operations to increase the receptive field of the network. Dilated TCN is composed of a series of blocks, where each block consists of a sequence of dilated temporal convolutional layers of increasing dilation rate. At early layers, a low dilation rate means filters capture short-term temporal dependencies. Later layers convolve the activations of early layers while simultaneously having larger dilation rates, capturing longer-range temporal

relations.

Both variants increase the temporal receptive field of the segmentation model significantly compared to previous approaches and are able to capture long-range temporal patterns. Additionally, TCN's can process all timesteps in the input in parallel, significantly decreasing training and inference time compared to models like RNN's, which can only process inputs sequentially.

2.1.2.3 Multi-Stage Temporal Convolutional Network

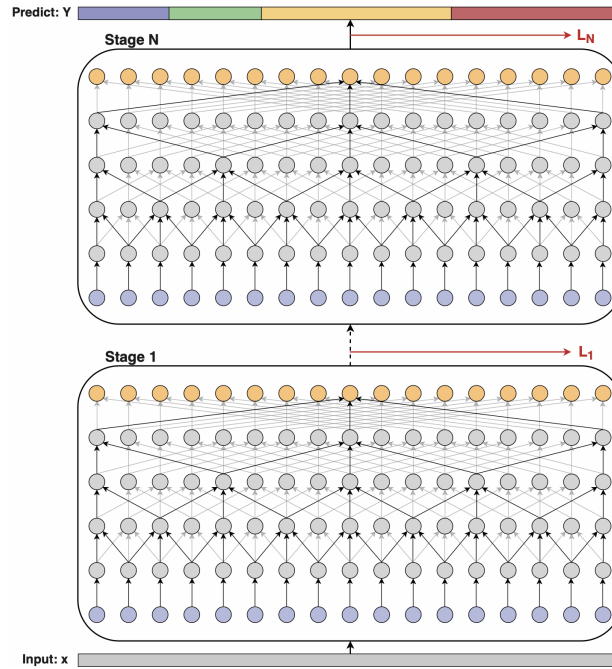


Figure 2.2: Overview of MSTCN. From [11].

Farha and Gall argue that the originally proposed ED-TCN method does not fully capture fine-grained details in two aspects. By using temporal pooling, some fine-grained details may be lost. The videos used for ED-TCN were also downsampled to only 1 – 3 frames per second, and the low temporal resolution might result in missing information at action boundaries or very short action segments. They propose in [11] the Multi-Stage Temporal Convolutional Network (MSTCN), an architecture that uses dilated temporal convolutional operations in a stack of stages to iteratively refine action class predictions.

Taking inspiration from the success of multi-stage architectures in human pose estimation, MSTCN (see Fig. 2.2) is composed of several stages, each of which takes

in the previous stage’s predictions and refines it. Each stage is similar to a block in Dilated TCN. It is composed of layers of dilated temporal convolutions that increase in dilation rate as the network gets deeper, achieving a large temporal receptive field with only a few kernel parameters. The first stage takes in the input features and outputs the initial frame-wise action class predictions. Successive stages takes the action class predictions of the prior stage and refines it. They hypothesize that the multi-stage architecture benefits TAS in two ways. The network is given more context when it predicts the action class at each timestep. The network is learning temporal dependencies between action class predictions instead of raw features, letting it learn action sequences better and prevent over-segmentation. They also introduce a novel smoothing loss that enforces the predicted logits of adjacent timesteps to be smooth.

Overall, their proposed method improved performance on all metrics when compared to previous approaches. Their smoothing loss and approach of multi-stage iterative refinement has been carried over into more recent works.

2.1.2.4 Attention-based TCN

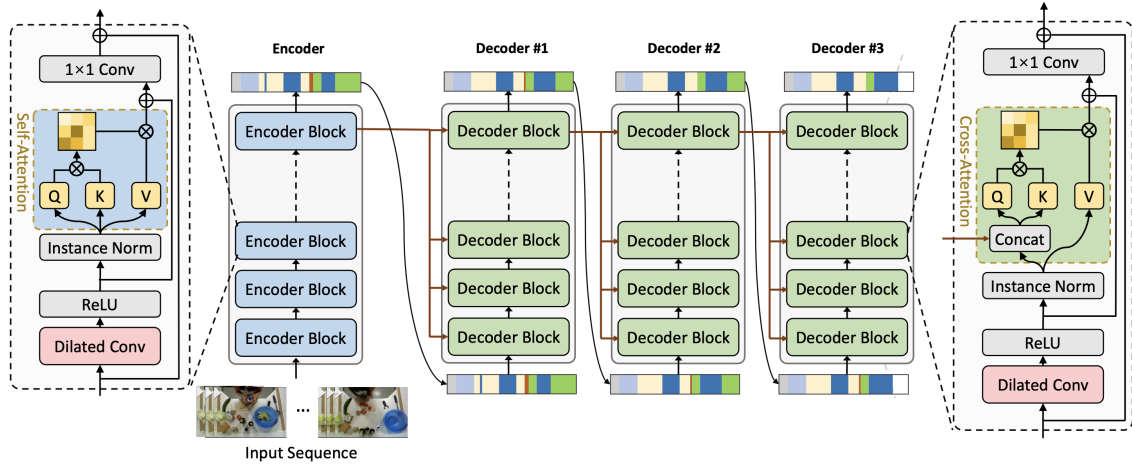


Figure 2.3: Overview of ASFormer. From [74].

The Transformer [66] architecture has seen great success in natural language processing and other sequence-related tasks. It has similarly been adapted into TAS by Yi et al. with the ASFormer [74].

The ASFormer (see Fig. 2.3) is a Transformer-inspired Encoder-Decoder network for TAS. The encoder block is composed of a sequence of layers of dilated temporal convolutions and self-attention mechanisms. These layers are hierarchical in that the dilation rate and self-attention operation’s window size are increased in as the layers go deeper, allowing the model to capture a larger temporal receptive field. The encoder’s outputs are used to calculate an initial per-frame action class prediction.

The decoder’s purpose is to refine the encoder’s initial action class predictions to improve performance. Similar to the encoder block, a decoder block is composed of a hierarchical sequence of dilated temporal convolutions and cross-attention mechanisms. Inspired by the success of MSTCN, the decoder follows a multi-stage design pattern. It is composed of multiple decoder blocks, where each block refines the action predictions of the previous block. The first decoder block takes in the initial encoder class predictions and cross-attends to the encoder’s output features. Later decoder blocks works with the previous decoder block’s predictions and output features. Each decoder block acts as a refinement stage that iteratively improves the model’s predictions.

The ASFormer was trained using a joint cross-entropy and smoothing loss, as was proposed in [11]. It was evaluated on standard TAS datasets and outperformed previous approaches, including MSTCN, on several metrics.

We experiment with MSTCN and ASFormer as the backbone networks of our approach.

2.2 Class Incremental Learning (CIL)

Incremental learning aims to develop algorithms that can continuously update a model with new information without forgetting prior information. Formulation of the incremental learning task can be categorized into three fundamental types [67]: task incremental learning, domain incremental learning, and class incremental learning.

In task incremental learning, the model is incrementally trained on discrete tasks, and the identity of the task is always available to the model, even during evaluation. In practice, this allows the model to use task-specific components. In domain incremental learning, tasks share the same labels, but the input experiences a domain shift. For example, the first task could be learning to classify a set of

landmarks at daytime, and the second task would be learning to classify the same landmarks but at night. In class incremental learning, the model is trained on a sequence of discrete tasks, and the task identity is not provided to the model. Effectively, the model needs to learn how to discriminate in growing number of classes, making this significantly more difficult than task incremental learning.

Incremental temporal action segmentation predicts action class labels for each input frame and is closest in nature to class incremental learning. The remainder of this work will discuss incremental learning in the context of class incremental learning (CIL).

2.2.1 Common Approaches

Catastrophic forgetting can be attributed to some primary factors [44]. When a model is trained incrementally, it is inclined to maximize performance on newer tasks. Weights and activations that were integral to old tasks are changed to minimize loss on the new task, leading to a form of network drift. Additionally, since the model is only exposed to samples of one task at a time, it is never jointly trained on all classes and suffers in discriminating classes between tasks, leading to inter-task confusion. Approaches that prevent catastrophic forgetting usually aim to mitigate these issues, and common techniques fall under either data replay or regularization.

2.2.1.1 Data Replay

Data replay or data rehearsal mitigates catastrophic forgetting by re-exposing the model to representatives of past data. A subset of the original training samples could be stored and used for review [50, 72, 18]. These methods can use a simple random sampling strategy or can use more considered approaches that select the most useful samples. The incremental classifier and representation learning (iCarl) [50] method uses herding to ensure that the average of the set of class samples it chooses is closest to the class’s training set average. Alternatively, the data distribution of past tasks could be learned by a model, and the exemplar set can be populated with generated samples [15, 34, 58]. The deep generative replay framework introduced in [58] trains a generative model alongside a classifier at each step. The generative model learns the cumulative input space up to the current task, and at the next incremental step, the generator is used to generate samples to train the next generator and

classifier.

2.2.1.2 Regularization

Another way to address catastrophic forgetting is to prevent model drift by adding regularization terms to the loss function. Weight regularization identifies the parameters in the model that are significant for past tasks and penalizes changes to them [25, 75, 2]. Kirkpatrick et al. propose elastic weight consolidation (EWC) which introduces a loss term that scales the penalty of a parameter update by an importance weight, such that changing the values of parameters that are important to past task predictions will incur a significant loss. Alternatively, instead of penalizing parameter updates, other methods have elected to minimize changes to layer activations [40, 76, 22]. In these methods, weights can be freely updated so long as the activations of past tasks remain similar. In Learning without Forgetting (LwF) [40], Li and Hoiem introduce a distillation loss term that encourages the temperature scaled prediction score of prior task classes to stay the same while training on a new task.

For this work, we propose a data replay approach for incremental temporal action segmentation. Data replay is commonly used and effective, and it proves to be a good initial approach for iTAS.

2.2.2 Data Replay in the Video Domain

A recent shift of incremental learning from static images to dynamic videos has brought about the application of data replay into the video domain, specifically action recognition [68, 3, 46, 47]. Due to the memory burden of storing full videos for data replay, the theme of CIL research in the action recognition has been the development of memory-efficient video data replay techniques.

Villa et al. propose in [68] to store temporally downsampled videos of past tasks in the replay buffer. They address the potential domain drift by introducing a temporal consistency loss to encourage the model to learn a similar feature representation for a video’s original and downsampled versions, i.e., learn representations that are invariant to temporal resolution.

Pei et al. propose in [47] a frame condensation approach called FrameMaker to reduce the storage demands of video exemplars by storing only a single representative frame for the entire video. FrameMaker takes in a video instance and integrates

CHAPTER 2. LITERATURE REVIEW

its frames to a condensed frame using learned frame-wise weights for each video instance. A set of learnable instance-specific prompt parameters are added to the condensed frame to enrich its representation capabilities. The authors interpret the prompt parameters as restoring the lost spatio-temporal information back into the condensed frame.

Alssum et al. similarly propose a single-frame exemplar approach in Single fraMe Incremental LEarner (SMILE) [3]. Instead of integrating a condensed frame like FrameMaker, SMILE selects a single frame from the video instance without modification and store it as the instance exemplar. To address the domain shift between the video and static image, they design a loss to encourage the model to learn similar representations for both the original video and its “boring” version, where a “boring” video is the sampled frame replicated temporally to match the original length.

Unlike current data replay approaches in action recognition, we use a generative model to populate the replay buffer, and unlike generative approaches for image tasks [15, 34, 58], our model incorporates a conditioning variable to account for the unique temporal coherence of videos. The coherence variable, defined as the relative progression within an action, assists the model in capturing the evolution of action features over time.

Chapter 3

Proposed Methodology

3.1 Problem Formulation

3.1.1 Temporal Action Segmentation

An untrimmed video $x^{1:T} = (x^1, \dots, x^T)$ is given, where T is the total number of frames of the video. x^t is the feature vector of the frame at time t . A model $\mathcal{M}$ is trained to separate the video into a set of N contiguous and non-overlapping action segments:

$$\begin{aligned} \mathbf{s}_{1:N} &= (s_1, s_2, \dots, s_N), \quad \text{where } s_n = (a_n, t_n, \ell_n), \\ \text{s.t.} \quad t_{n+1} &= t_n + \ell_n, \end{aligned} \tag{3.1}$$

where $a_n \in \mathcal{A}$ is the action class label assigned to the segment chosen from $\mathcal{A}$ predefined classes, t_n denotes the starting timestamp of the segment, and ℓ_n is the segment duration. t_n adheres to a precise temporal sequence from the preceding segment. This formulation of the problem is more commonly seen in approaches dealing with weak forms of supervision [54, 53, 35].

Alternatively, the training objective of the model can be reformulated as a per-frame classification task. The model $\mathcal{M}$ can be trained to produce:

$$y^{1:T} = (y^1, \dots, y^T), \tag{3.2}$$

where $y^t \in \mathcal{A}$ is the predicted action label for frame at time t . Most of the recent works [74, 10, 61, 42, 37] adopt this per-frame formulation of the task. Our approach similarly produces per-frame action class predictions.

The segmentation model $\mathcal{M}$ is trained with a frame-wise cross-entropy loss:

$$\mathcal{L}_{\text{cls}}(x, y) = \frac{1}{T} \sum_t -\log(p^t(y^t)), \quad (3.3)$$

where $p^t \in \mathbb{R}^A$ is the estimated action probability for the frame x^t . In addition, a smoothing loss is imposed to ensure smooth transitions between consecutive frames:

$$\mathcal{L}_{\text{sm}}(x, y) = \frac{1}{TA} \sum_{t,a} \tilde{\Delta}_{t,a}^2, \quad \tilde{\Delta}_{t,a} = \begin{cases} \Delta_{t,a} : \Delta_{t,a} \leq \tau \\ \tau & : \text{otherwise} \end{cases}, \quad (3.4)$$

$$\Delta_{t,a} = \left| \log p^t(a) - \log p^{t-1}(a) \right|.$$

τ is conventionally set to 4, as per [11]. The full training loss is written as the combination of the above two, balanced by hyperparameter λ :

$$\mathcal{L}_{\text{tas}} = \mathcal{L}_{\text{cls}} + \lambda \cdot \mathcal{L}_{\text{sm}}. \quad (3.5)$$

3.1.2 Class Incremental Learning

In class incremental learning, classes are gradually introduced to model $\mathcal{M}$ through a series of B tasks $\{\mathcal{T}_1, \dots, \mathcal{T}_B\}$. Each task $\mathcal{T}_b$ represents a distinct incremental step, comprising a set of n_b training instances denoted as $\{(x_i, y_i)\}_{i=1}^{n_b}$. Here, x_i represents an instance associated with class $y_i \in \mathbf{Y}_b$, where $\mathbf{Y}_b$ is the class set for task b . Importantly, access to the training data in $\mathcal{T}_b$ is restricted to current training task b only. The model cannot access data from previous or future tasks. It must acquire knowledge from the current task while preserving knowledge from the past tasks. The performance of an incrementally trained model is evaluated over all seen classes, i.e., $\mathcal{Y}_B = \mathbf{Y}_1 \cup \dots \mathbf{Y}_B$. Standard CIL assumes non-overlapping classes in different tasks ($\mathbf{Y}_b \cap \mathbf{Y}_{b'} = \emptyset$ for $b \neq b'$). Blurry CIL is an alternate configuration that allows for classes between tasks to overlap [26, 4, 5].

In data replay, a model is allowed to revisit exemplars from prior tasks via a replay buffer. The replay buffer, also known as an exemplar set, is an extra collection of instances from the previous tasks $\hat{\mathcal{T}}_{1:b} = \{(x_{b'}, y_{b'})\}_{b'=1}^{b-1}$, $y_{b'} \in \mathcal{Y}_{b-1}$. The exemplars can either be previously seen training samples [50], constructed prototypes [20], or generated samples [58, 73]. The training instances in the joint set $\mathcal{T}_b \cup \hat{\mathcal{T}}_{1:b}$ can then be utilized to update model $\mathcal{M}$ with new classes from task $\mathcal{T}_b$ while retaining previous knowledge via exemplars in $\hat{\mathcal{T}}_{1:b}$.

3.1.3 Incremental Temporal Action Segmentation

In incremental temporal action segmentation (iTAS), a segmentation model $\mathcal{M}$ is continuously updated by introducing it to new action classes at each incremental step. Since procedural videos from the same activity (e.g. preparing “*coffee*”) share a common set of interrelated actions (e.g. ‘*pour coffee*’, ‘*pour milk*’), we consider each procedural activity as an incremental task b , and the actions within each activity as class labels $\mathbf{Y}_b$. The segmentation model $\mathcal{M}$ learns to segment videos from various incremental activities. To address the high spatial cost of naively storing previous video training samples in the exemplar set, our approach introduces a novel generative data replay approach for iTAS featuring a temporally coherent action model.

3.2 Generative Data Replay with a Temporally Coherent Action Model

3.2.1 Temporally Coherent Action (TCA) Model

Generative models, such as Variational Autoencoders (VAE) [24] and Generative Adversarial Networks [14] are powerful tools for learning efficient representations of data with a fixed model size. Additionally, they are easy to extend to a conditional form to generate conditioned outputs. These properties make them a good fit for the purpose of generating diverse action segments for data replay.

In this work, we employ a conditional VAE [62] to model the actions. Other generative models are also feasible, but conditional VAEs offer a good trade-off between model size, expressiveness, and data efficiency for learning. In a conditional VAE model, an encoder maps an input frame feature x and the conditioning action class label a to a probability distribution $q_\phi(z|x, a)$ over the latent space, where z is the latent variable. The decoder maps a sample from latent space z and the conditioning information back to obtain a reconstruction of the input $\hat{x} = p_\theta(x|z, a)$. ϕ, θ are learnable network parameters of the encoder and decoder, respectively. The overall loss function is a combination of the reconstruction term and the KL

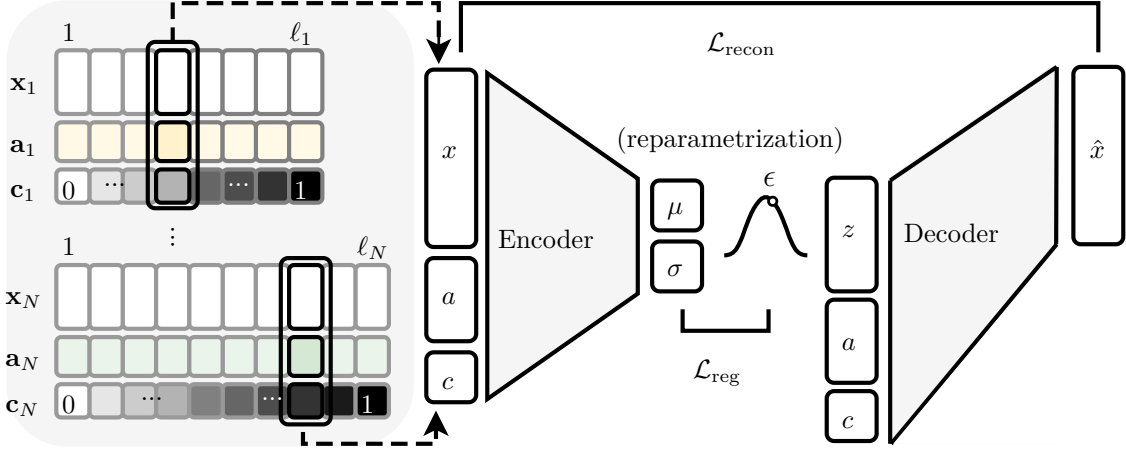


Figure 3.1: Temporally Coherent Action (TCA) model.

divergence regularization term, written as:

$$\mathcal{L}_{\text{cVAE}} = \underbrace{\mathbb{E}_z \log p_\theta(x|z, a)}_{\text{reconstruction}} - \underbrace{\text{D}_{\text{KL}}(q_\phi(z|x, a)||p(z))}_{\text{regularization}}$$

The second term regularizes the latent space by a prior $p(z)$ on the approximate posterior distribution; typically, the prior is a Gaussian distribution.

While the above action model captures the diversity of action segments, it ignores any temporal coherence since the action frames are modeled independently. To that end, we introduce a coherence variable to model the temporal transition between feature frames within an action segment. This work formulates the coherence variable as the relative temporal progression of a frame within the action. For the i -th frame in an action segment of duration ℓ , the coherence variable c_i is defined as:

$$c_i = (i-1)/(\ell-1), \text{ and } c_i \in [0, 1]. \quad (3.6)$$

Incorporating the relative progression as the coherence condition helps the model build up knowledge of the feature continuity following the progression of actions. In this work, we follow [6] and assume the action progression to be linear.

Adding upon Section 3.2.1, the training loss for our Temporally Coherent Action (TCA) model is written as:

$$\mathcal{L}_{\text{TCA}} = \underbrace{\mathbb{E}_z \log p_\theta(x|z, a, c)}_{\mathcal{L}_{\text{recon}}} - \underbrace{\text{D}_{\text{KL}}(q_\phi(z|x, a, c)||p(z))}_{\mathcal{L}_{\text{reg}}}. \quad (3.7)$$

Fig. 3.1 illustrates the TCA model. For each frame in a segment, the encoder $\mathcal{E}(x, a, c) = q_\phi(z|x, a, c)$ takes in the feature x , one-hot action label a , and its coherence variable c , while the decoder $\mathcal{D}(z, a, c) = p_\theta(x|z, a, c)$ samples a latent variable z with reparametrization: $z = \mu + \sigma \odot \epsilon, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{1})$ and uses the same a, c for reconstruction.

3.2.2 Replay Data Generation

Our replay data generation procedure is top-down. We first sample the sequential structure of action segments to generate. We then populate the action segments with temporally coherent frames from the TCA model.

3.2.2.1 Sequential Structure Sampling

In order to sample the sequential structure of the video to generate, we refer back to the segment-level interpretation provided by Eq. (3.1). A procedural video can be summarized as an ordered sequence of actions, each with varying duration. Such a high-level structure helps guide the data generation process. The (symbolic) sequences require negligible storage, so we keep all the sequences from the training data and directly sample from the set during generation. Specifically, we establish a candidate pool to store all action sequence order and their duration $\mathbf{S}_b = \{\mathbf{s}_i\}_{i=1}^{n_b}, \mathbf{s}_i \in \mathcal{T}_b$. During the data generation stage, we employ a uniform sampling strategy on $\mathbf{S}_b$:

$$\hat{\mathbf{s}}_b \sim \text{Uniform}(\mathbf{S}_b). \quad (3.8)$$

3.2.2.2 Action Segment Generation.

The sampled sequential structure $\hat{\mathbf{s}} = \{(a_n, t_n, \ell_n)\}_{n=1}^{\hat{N}}$ is then utilized in the segment generation process, as illustrated in Fig. 3.2. For each sampled (a_n, t_n, ℓ_n) , our TCA model generates the frame-wise features for the segment using:

$$\hat{x}_i = p_\theta(x|z_n, a_n, c_i), \text{ and } i \in [1, \dots, \ell]. \quad (3.9)$$

Here, we fix z_n across all frames for the given action a_n and vary the coherence variable c_i , following Eq. (3.6). This ensures temporal coherence in feature transition within the segment. The process is repeated for every segment, and these generated segment features $\hat{\mathbf{x}}_n$ are concatenated according to their timestamps t_n to compose

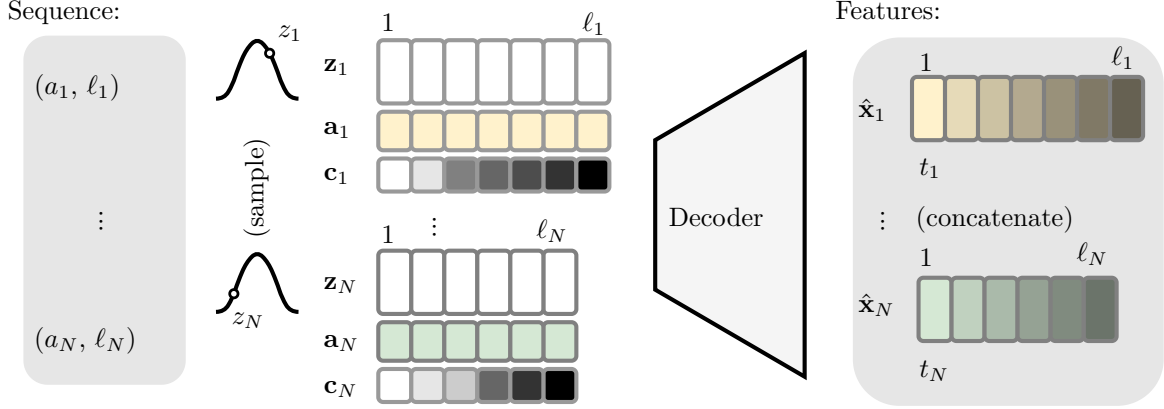


Figure 3.2: Replay Data Generation.

the procedural video:

$$\hat{v} = \text{concat}(\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_{\hat{N}}). \quad (3.10)$$

3.2.2.3 Discussion

An intuitive interpretation of the latent and conditioning variables in Eq. (3.9) can be as follows: z governs the overall scene context, a regulates the action semantics, and c manages the temporal progression of the action features. Since the action label a is predefined for each segment, it is kept consistent.

Other generation options for the action segments can be achieved by manipulating the latent variable z and the conditioning coherence variable c . We use this flexibility to conduct an ablation study in Section 4.5.1 regarding the necessity of feature diversity and temporal coherence in video data replay.

We ensure temporal coherence within segments but not across action transitions because boundary frames, as indicated in [64, 10], are ambiguous and may not be conducive to learning the segmentation model.

3.2.3 iTAS Training

We commence by training the segmentation model $\mathcal{M}$ (Fig. 3.3(a)) and our TCA model $(\mathcal{E}_1, \mathcal{D}_1)$ with the initial task data $\mathcal{T}_1$. As new task data $\mathcal{T}_b$ becomes available, we create replay video data $\hat{v}$ for all previous tasks $[1:b)$ utilizing their corresponding TCA decoders $\mathcal{D}$. This process yields a total of M videos denoted

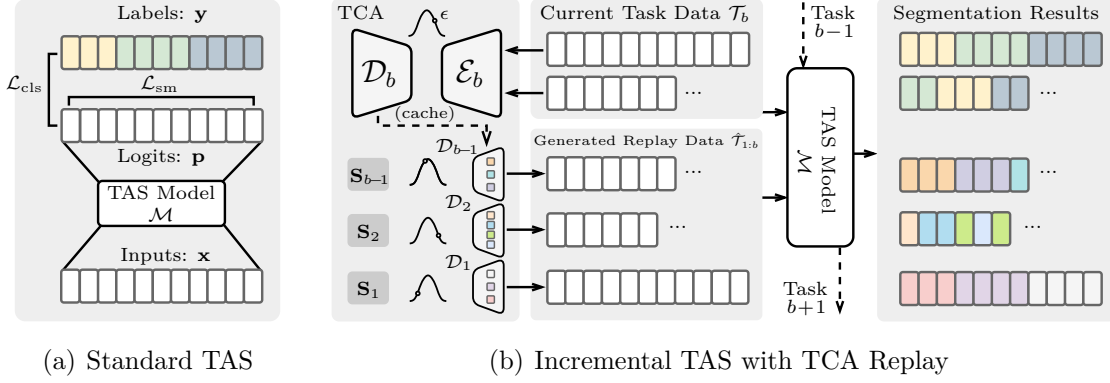


Figure 3.3: Learning schemes for temporal action segmentation.

Algorithm 1 Incremental Temporal Action Segmentation

Input: Task video data $\{\mathcal{T}_1, \dots, \mathcal{T}_B\}$, replay size M
Output: Segmentation model $\mathcal{M}$

- 1: Train $\mathcal{M}$ with data $\mathcal{T}_1$; $\triangleright$ Eq. (3.5)
 - 2: Train $(\mathcal{E}_1, \mathcal{D}_1)$ with data $\mathcal{T}_1$; $\triangleright$ Eq. (3.7)
 - 3: **for** $b = 2, \dots, B$ **do** $\triangleright$ *Incremental Learning*
 - 4: Initialize replay data $\hat{\mathcal{T}}_{1:b} = \emptyset$;
 - 5: **for** $b' = 1, \dots, b-1$ **do** $\triangleright$ *Replay Data Generation*
 - 6: Get $\hat{\mathbf{s}}_{b'}$ from $\mathbf{S}_{b'}$ for $M/(b-1)$ times; $\triangleright$ Eq. (3.8)
 - 7: Generate $\hat{v}_{b'}$ for each $\hat{\mathbf{s}}_{b'}$ with $\mathcal{D}_{b'}$; $\triangleright$ Eq. (3.10)
 - 8: $\hat{\mathcal{T}}_{1:b} \leftarrow \hat{\mathcal{T}}_{1:b} \cup \{\hat{\mathbf{v}}_{b'}\}_{M/(b-1)}$;
 - 9: **end for**
 - 10: Train $\mathcal{M}$ with data $\mathcal{T}_b \cup \hat{\mathcal{T}}_{1:b}$; $\triangleright$ Eq. (3.11)
 - 11: Train $(\mathcal{E}_b, \mathcal{D}_b)$ with data $\mathcal{T}_b$; $\triangleright$ Eq. (3.7)
 - 12: **end for**
-

as $\hat{\mathcal{T}}_{1:b}$. To train the segmentation model $\mathcal{M}$, we combine this generated data with the real data from $\mathcal{T}_b$ to serve as the training set, as shown in Fig. 3.3(b). Through this, the segmentation model can revisit the previous tasks to mitigate catastrophic forgetting while learning new tasks. The incremental learning objective at task b is as follows:

$$\mathcal{L}_{\text{iltas}}^b = \underbrace{\mathcal{L}_{\text{cls}}(x, y)}_{(x, y) \in \mathcal{T}_b \cup \hat{\mathcal{T}}_{1:b}} + \lambda \cdot \underbrace{\mathcal{L}_{\text{sm}}(x, y)}_{(x, y) \in \mathcal{T}_b \cup \hat{\mathcal{T}}_{1:b}}, \quad (3.11)$$

we set $\lambda = 0.15$, following [11]. Concurrently, as iterating through the above steps, we train our TCA model $(\mathcal{E}_b, \mathcal{D}_b)$ every time new video data from task b arrives. The overall incremental learning procedure is summarized in Algorithm 1.

Chapter 4

Experiments

4.1 Datasets

We adapt Breakfast [28], CrossTask [77], and YouTube Instructional [1] datasets to the incremental learning setup. Note other TAS datasets such as GTEA [12] and 50Salads [65] are unsuitable because most of their videos share a common set of actions, making it challenging to implement a meaningful number of incremental tasks without a significant action overlap.

Dataset	# Videos	Duration	# Activities	# Actions	Domain
Breakfast [28]	1712	77h	10	48	Cooking
CrossTask [77]*	2140	170h	14	112	Mixed
YTI [1]	150	7h	5	46	Mixed

*CrossTask processed as described in Section 4.1.2.

Table 4.1: Summary of iTAS Datasets.

4.1.1 Breakfast

Breakfast [28] dataset is composed of 1,712 cooking videos, totalling to approximately 77 hours of footage. It consists of 52 participants performing everyday breakfast preparation tasks. The videos were recorded in 18 different kitchen settings from multiple camera views. Participants were completely undirected. Thus, the dataset contains significant variability in both action choice and action sequence order, making it a challenging benchmark in TAS.

There are 10 cooking activities and a total of 48 action classes. Each activity features 5 to 14 actions and comprises around 150 videos for training and 20–30 for

testing. Each video contains 6.8 action segments on average. We use the I3D [7] feature representations and evaluate with the standard splits.

4.1.2 CrossTask

CrossTask [77] is a large dataset composed of 4,713 videos demonstrating 83 activities. Of these activities, only the 18 activities categorized as *primary* are fully annotated with temporal locations of each action segment, while the remaining 65 activities are not. We adapt the 18 primary activities into incremental temporal action segmentation.

Within the 18 primary activities, some activities have significant overlap with others. For example, 80% of the actions in “*make irish coffee*” is shared with activities like “*make latte*” and “*make jello shots*”. We manually filter out activities with a large overlap to maintain a level of distinctiveness between tasks. After filtering, the CrossTask dataset we use is composed of 2,140 videos totalling to around 170 hours of footage. There are 14 activities and 112 actions. Each activity contains 6 to 12 actions. Each video has 14.6 action segments on average.

We use 80% of the videos in each activity for training and reserve the remaining for evaluation. We use the feature representation designed by the authors of the dataset, which is concatenated from the RGB I3D [7], ResNet-152 [16], and audio [17] features extracted from one second temporal windows.

4.1.3 YouTube Instructional

YouTube Instructional (YTI) [1] dataset is composed of 150 narrated instructional videos collected from YouTube, totalling to around 7 hours of footage. The dataset comprises of 5 activities with 30 video samples for each activity. There are a total of 46 actions with each activity featuring 6 to 13 actions. Each video sample contains 17.2 action segments on average.

Similar to CrossTask, we use 80% of the videos in each activity for training and reserve the remaining for evaluation. We use the same set of feature representations as [57, 31], which is formed by concatenating motion (bag-of-visual-word HOF [70]) and appearance (VGG-16 [59]) features.

4.1.4 Adapting for iTAS

We partition the Breakfast, CrossTask, and YTI datasets based on activities. Each activity is a separate task, and we train models incrementally on them. We experiment with both disjoint and blurry incremental settings on Breakfast.

Disjoint tasks consider shared actions different, e.g., ‘*take plate*’ in “*friedegg*” is a different class from ‘*take plate*’ in “*sandwich*”. In the blurry setup for Breakfast, overlapping tasks are allowed, such that common actions between tasks are the assigned the same class label.

Breakfast, CrossTask, and YTI all contain background frames, which are frames that do not depict the task taking place (e.g. narration). We treat these background frames as another action class and include them during training and evaluation.

4.2 Evaluation Metrics

Temporal action segmentation is evaluated by a frame-wise accuracy (Acc), segment-wise edit score (Edit), and F1 score with varying overlap thresholds of 10%, 25%, and 50% [9]. We adopt these standard measures and apply them to the incremental setup.

4.2.1 Frame-wise Metrics

Frame-wise accuracy (Acc), also known as Mean over Frames in TAS literature, is the percentage of the model’s correct frame predictions, i.e.,

$$\text{Acc} = \left(\frac{\# \text{ of correct predictions}}{\# \text{ of predictions}} \right) * 100 \quad (4.1)$$

It gives a good measure of the model’s action classification capabilities. However, it fails to capture the segment quality in the model’s output. Over-segmentation is a phenomenon where an action segment is fragmented into discontinuous sub-segments by erroneous frame predictions. Though a model’s predictions may exhibit over-segmentation, a high Acc score can still be reported if the incorrect predictions are comparatively few and interspersed. To capture the quality of the segments in a model’s output, we must also evaluate the predictions on segment-wise measures.

4.2.2 Segment-wise Metrics

The edit score [33] is a segment-wise metric based on the Levenshtein edit distance. The edit distance $e[|X|, |Y|]$ between two action sequences X and Y is given by the minimum number of edit operations (i.e. insertion, deletion, replacement) required to transform one sequence to another. The $e[|X|, |Y|]$ distance is normalized to get the edit score, i.e.,

$$\text{Edit} = \left(1 - \frac{e[|X|, |Y|]}{\max(|X|, |Y|)}\right) * 100 \quad (4.2)$$

The edit score is a useful metric that evaluates the model’s ability to predict action sequence order without needing exact temporal alignment with the ground truth.

The F1 score or F1@ τ [32] is another segment-based metric and is built upon Intersection-over-Union (IoU) scores. A predicted action segment is considered a true positive of the ground truth segment if its IoU score exceeds $\frac{\tau}{100}$ and if their action labels match. Only the predicted segment with the highest IoU score in the span of the ground truth segment is considered a true positive, and any other predicted segments in the same span are marked as false positives. Any missed ground truth segments are false negatives. From these results, the F1 score can be calculated as the harmonic mean of the precision and recall scores, i.e.,

$$\text{F1} = 2 \cdot \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}} \quad (4.3)$$

Compared to the edit score, the F1 score is stricter and can more heavily penalize over-segmentation errors.

4.2.3 Evaluation in iTAS

For iTAS, we denote the model’s Acc on task b' after being trained on the b -th task as $\text{Acc}_b^{b'}$, where $b' < b$. We use the last stage accuracy averaged over all tasks as the final metric to measure the overall performance, i.e.,

$$\text{Acc} = \frac{\sum_{b=1}^B \text{Acc}_B^b}{B}.$$

We calculate the average performance over all tasks due to the imbalance in frame numbers across each task. The final reported Edit and F1 scores are similarly defined, i.e.,

$$\text{Edit} = \frac{\sum_{b=1}^B \text{Edit}_B^b}{B},$$

$$\text{F1} = \frac{\sum_{b=1}^B \text{F1}_B^b}{B}.$$

4.3 Implementation Details

4.3.1 TCA Model

Our TCA model comprises an encoder and decoder; each implemented as a two-layer MLP with a 256-D latent space. We train TCA for 2,500 epochs with batch size 512 for Breakfast, 250 epochs with batch size 64 for YTI, and 250 epochs with batch size 32 for CrossTask. The learning rate is set to be $1e^{-3}$, $1e^{-5}$, and $1e^{-4}$, respectively.

4.3.2 TAS Backbones

We experiment with the temporal convolutional network MSTCN [11] and the transformer-based ASFormer [74] for our backbone model $\mathcal{M}$. MSTCN is trained with a learning rate of $5e^{-4}$ for 50 epochs for each task; for ASFormer, it is $1e^{-4}$ for 30 epochs. Both models are trained with the ADAM optimizer.

4.3.3 Replay Size

For all experiments, we constrain the maximum number of videos for replay to 60, which is equally divided among the seen tasks. The value of 60 was chosen to maintain a similar exemplar-to-training ratio (1:25) per iCIFAR-100 used in [50].

Given that the action sequence of generated videos is sampled from past task video instances, the total number of frames generated as the replay buffer has an upper limit of $60 \times \text{longest video}$. Our ablation study (see Section 4.5.2) shows that the performance can be further improved with a larger buffer size.

4.4 Effectiveness

4.4.1 Baselines

We first use a naive (**‘Finetune’**) baseline that progressively finetunes with only training data in the task sequence without data replay. Additionally, we follow the boring baseline from action recognition [3] and store a mean frame for each action segment per sequence, which we call **‘Exemplar’**. Such a segment is static, as the frame-wise sample is simply replicated to inflate the segment in time and used for data replay. Finally, we consider an upper-bound (**‘Original’**) data replay baseline that uses the original sequence features.

4.4.2 Results

# Tasks		MSTCN [11]					ASFormer [74]				
		Acc	Edit	F1 @ {10, 25, 50}			Acc	Edit	F1 @ {10, 25, 50}		
Breakfast											
10	Finetune	7.4	7.2	7.5	7.0	5.4	9.9	9.8	10.3	9.4	7.5
	Exemplar [3]	16.1	13.3	13.8	12.5	9.5	12.4	11.2	11.7	10.7	8.5
	Ours	29.4	25.9	26.3	23.5	17.7	34.2	32.4	33.1	30.1	23.4
	Original	43.1	41.1	41.2	37.6	29.5	48.1	45.2	45.9	42.4	34.2
5	Finetune	15.4	15.8	16.6	15.8	12.7	15.7	16.1	16.9	15.8	13.2
	Exemplar [3]	32.5	28.9	30.8	28.5	22.9	29.5	27.5	28.7	26.7	22.0
	Ours	54.5	49.4	51.1	46.9	37.7	57.2	56.8	58.3	54.0	43.6
	Original	60.4	59.1	60.3	56.1	46.0	65.1	64.2	65.6	61.5	51.0
CrossTask											
14	Finetune	5.3	3.8	3.8	3.4	2.5	5.3	4.1	3.9	3.5	2.5
	Exemplar [3]	9.8	6.7	6.5	5.5	3.6	12.2	8.5	8.6	7.4	5.0
	Ours	35.4	22.9	22.7	19.4	12.4	46.5	32.2	31.3	27.3	18.0
	Original	40.8	26.9	26.6	23.4	15.7	48.5	30.5	31.2	27.5	18.3
YouTube Instructional											
5	Finetune	13.6	2.8	3.6	2.7	0.6	13.9	11.5	11.1	9.8	6.3
	Exemplar [3]	30.8	19.7	19.8	16.0	9.3	22.1	18.9	17.7	15.3	10.0
	Ours	30.2	25.0	21.9	18.5	11.1	25.2	20.9	20.1	17.5	11.4
	Original	55.9	39.4	38.1	32.2	19.1	59.2	51.1	45.4	39.1	25.5

Table 4.2: Performance comparison on Breakfast, CrossTask, and YouTube Instructional.

We compare the performances on three datasets in Table 4.2. We adopt the approach of prior incremental learning studies by initializing the task sequence

with multiple random seeds. We utilized five specific random seeds throughout our experiments: {42, 123, 1000, 1993, 2023}. The performance results presented reflect the mean across multiple splits and seeds.

Finetune achieves the lowest performance as it simply concentrates on learning new concepts without revisiting previous tasks. In the 10-task setup on Breakfast, static segments with stored exemplar features mitigate forgetting, gaining an accuracy increase from 7.4% to 16.1% with MSTCN. Our approach achieves a more substantial boost, reaching 29.4%. The improvement is consistent across both backbones. Despite this improvement, there remains a 13.1% gap compared to using original features from previous tasks.

There are similar performance differences across the approaches in the 5-task incremental Breakfast setup. The 5-task incremental setup combines every two activities as a single task and has fewer training steps, improving performance across all approaches. This suggests that the challenge of learning increases with the number of tasks, leading to increased forgetting by the model.

For CrossTask, our approach likewise attains impressive performance gains in all metrics compared baselines on both backbones. Accuracy jumps from the Exemplar baseline’s 9.82% to 35.41% with MSTCN and from 12.22% to 46.54% with ASFormer. The edit and F1 scores also show similar improvements.

The performance gains for the YTI dataset is less significant compared to either Breakfast or CrossTask; both the Exemplar [3] and our method achieve very similar performances. We hypothesize that this phenomenon results from the overall reduced segment diversity in the YTI dataset due to its comparatively smaller size. Nevertheless, the rise in the segmental metrics suggests that temporally evolving action segment features, as opposed to static ones, can alleviate over-segmentation.

	MSTCN [11]					ASFormer [74]				
	Acc	Edit	F1 @ {10, 25, 50}			Acc	Edit	F1 @ {10, 25, 50}		
Finetune	15.8	29.7	30.0	26.0	19.1	15.5	30.2	30.6	26.4	19.6
Exemplar [3]	25.0	34.0	34.9	30.2	22.3	24.4	35.6	37.2	32.9	24.9
Ours	38.5	43.3	44.9	39.5	29.7	44.2	51.2	53.0	47.6	36.8
Original	48.5	53.4	55.1	49.4	37.9	53.5	57.6	59.9	54.5	43.2

Table 4.3: Performance comparsion on Breakfast with the blurry task boundary.

Our evaluation using the blurry setup on the Breakfast dataset is presented in Table 4.3. We consistently observe performance improvements, with all performance metrics surpassing the standard setup (Table 4.2) by 10 points. This improvement is likely attributed to the efficient updating of action classifiers from previous tasks with the current task samples due to shared actions and blurred task boundaries.

4.5 Ablation Studies

4.5.1 Feature Diversity and Temporal Coherence

	SD	FD	TC	Acc	Edit	F1 @ {10, 25, 50}		
Exemplar	✓	✗	✗	27.8	35.6	36.1	31.7	24.3
Ours _{random}	✓	✓	✗	32.9	38.9	40.0	35.6	27.2
Ours _{static}	✓	✗	✗	37.9	42.9	43.8	38.9	29.0
Ours	✓	✓	✓	41.8	45.0	47.0	41.5	32.0

Table 4.4: Ablation study on the components in TCA.

In our evaluation of the impact of feature diversity and temporal coherence on performance (see Table 4.4), we assess three key factors: the diversity of actions at the segment level (‘SD’), the diversity of frames within a segment (‘FD’), and the temporal coherence of a segment (‘TC’). The gray row indicates the final setup used in this work. For this ablation study, we define two additional setups, Ours_{static} and Ours_{random}, by manipulating the z and c variables used in the TCA model when generating action segments.

In Ours_{static}, we generate a *static* segment with a constant latent variable z and a constant coherence c spanning the entire segment. The setup is similar to Exemplar in that a single feature vector is repeated to span the entire segment. In Ours_{random}, we generate a dynamic yet *random* segment by fixing c but having z_i independently sampled for each frame. Feature diversity is maximized, but no temporal constraints are enforced.

Both with static segments, Ours_{static} outperforms the Exemplar by approximately 10% in accuracy (Rows 1 *vs.* 3). The Exemplar baseline stores the mean frame of each action segment for replay. These features will remain identical across training

increments. Meanwhile, $\text{Ours}_{\text{static}}$ generates new static features to populate the action segment at each incremental step. The 10% improvement showcases TCA’s ability to generate diverse actions and enhance segment-level diversity. However, $\text{Ours}_{\text{random}}$ performs less effectively than $\text{Ours}_{\text{static}}$, highlighting the detrimental impact the absence of temporal constraints can cause for the TAS task. In the final row, our approach, which coordinates feature diversity and temporal coherence, achieves the highest performance.

4.5.2 Replay Size

M	Acc	Edit	F1 @ {10, 25, 50}		
30	34.0	39.6	41.0	34.8	24.7
60	35.4	41.2	42.3	36.0	25.6
90	36.2	42.3	43.9	37.3	26.8
120	38.0	42.3	44.0	37.1	26.2

Table 4.5: Different replay sizes on Breakfast.

We examine four different replay sizes: $M = 30, 60, 90, 120$. As depicted in Table 4.5, performance is observed to be correlated with the replay size M as anticipated. Employing only 30 video sequences as replay data results in a marginal decline (34.0%) compared to 60 (35.4%). An increase in the replay size M corresponds to incremental improvements in both frame-wise and segmental metrics, and the best performance is achieved with the replay size being set to 120.

4.5.3 TCA Training Data

	$\mathcal{T}(\%)$	Acc	Edit	F1 @ {10, 25, 50}		
Exemplar	-	22.6	34.8	36.0	32.4	25.2
Ours	25	41.7	43.2	46.1	40.9	31.5
	50	42.1	43.3	45.1	40.5	31.5
	75	45.3	45.9	47.8	43.7	34.7
	100	47.4	46.9	48.2	42.8	33.4

Table 4.6: Different ratios for TCA training on Breakfast.

The results of training TCA with varying proportions of task data $\mathcal{T}$ are presented in Table 4.6. A consistent trend indicates that increased access to task data during TCA training brings greater overall performance improvements and less forgetting. Training TCA with the entire task data yields the overall best performance, with a significant 6% decrease in performance observed when only a quarter of the data is utilized. Notably, our approach outperforms the Exemplar significantly, even when trained with only 25% of task data.

4.5.4 Task Sequence

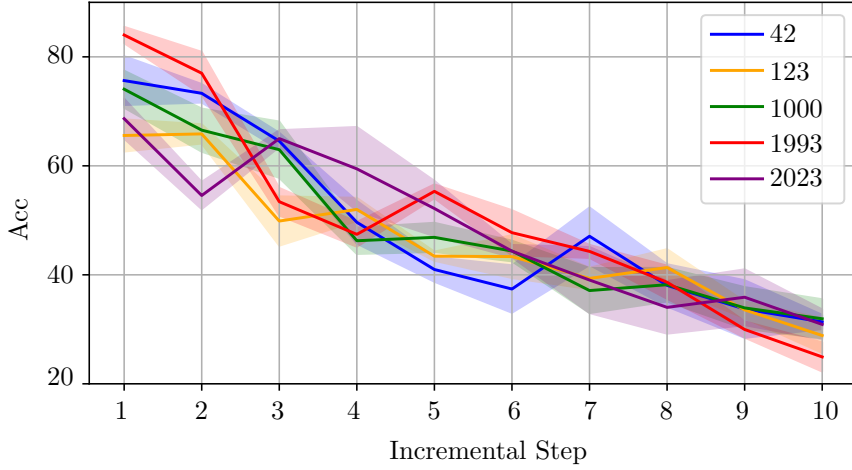


Figure 4.1: Performance following each incremental step with the varied task sequence.

We compare five distinct task sequence arrangements within a 10-task incremental setup on Breakfast and report the results in Fig. 4.1. The mean and standard deviation across four splits are represented by solid lines and shaded areas, respectively. Each unique task sequence, determined by a random seed, is distinguished by a different color.

Although there is a consistent decrease in accuracy scores, the ultimate performance is subject to variation depending on the seed used for task sequence determination, showing differences of up to 7% points. This suggests that the forgetting in incremental learning is related to the order of these learning tasks. Moreover, the disparity could be exacerbated by the uneven distribution of frames across tasks, which is commonly observed in procedural videos.

4.5.5 Visualizations

4.5.5.1 Confusion Matrix

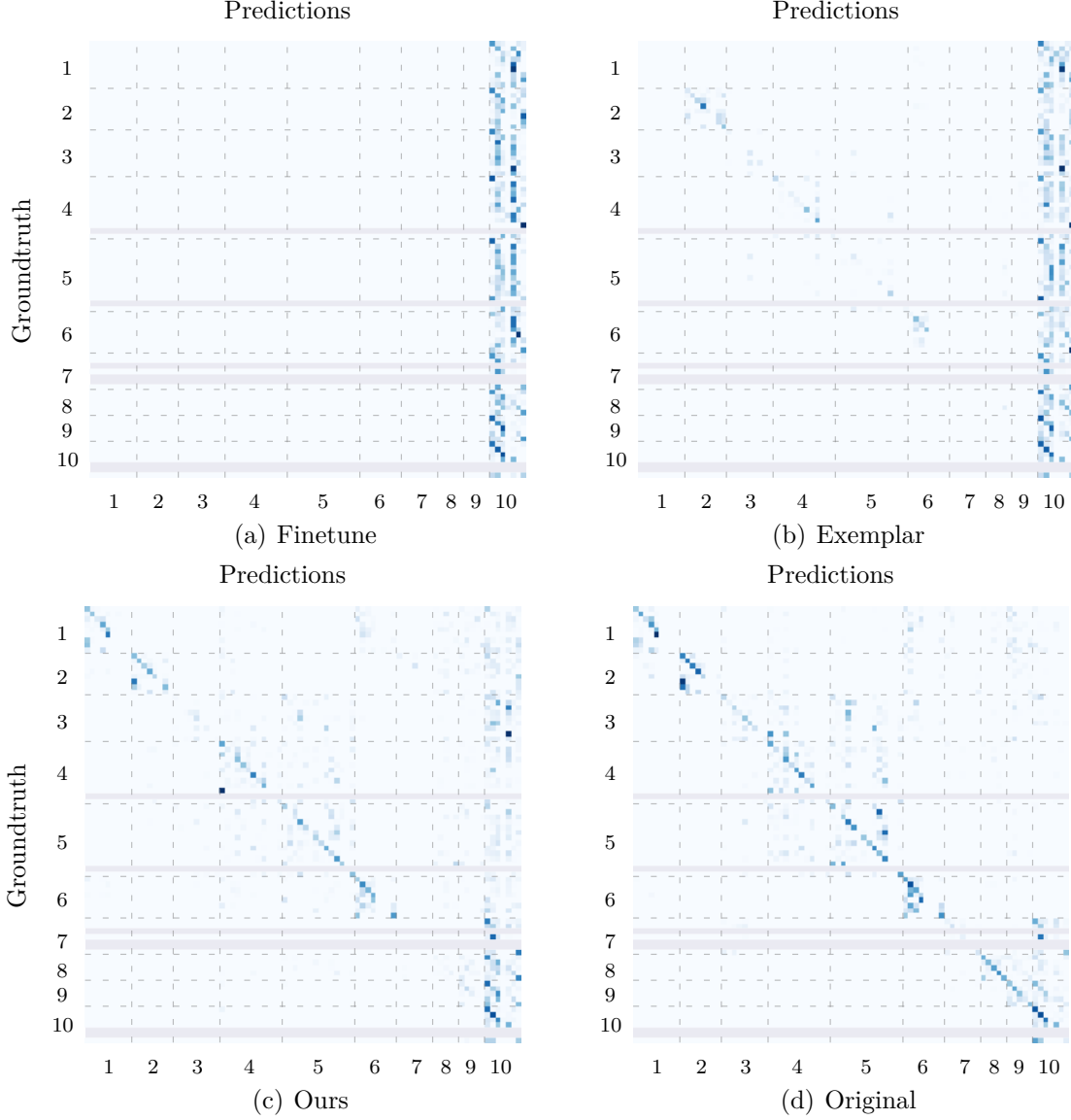


Figure 4.2: Comparison of confusion matrices of different approaches on Breakfast in a 10-task setup.

In Fig. 4.2, we plot the confusion matrices generated by different approaches on the 10-task Breakfast setup. The task sequence is given as follows: 1 - "sandwich", 2 - "juice", 3 - "friedegg", 4 - "scrambledegg", 5 - "pancake", 6 - "salat", 7 - "tea", 8 - "milk", 9 - "cereal", 10 - "coffee". The dashed lines indicate task boundaries, while

gray rows denote the lack of instances for that action in the test data.

Our method demonstrates superior performance compared to Finetune and Exemplar, particularly in the case of longer activities such as “*sandwich*”, “*scrambledegg*”, “*pancake*” and “*salat*”. Furthermore, similar to Original, our approach exhibits increased confusion between semantically similar activities, such as “*scrambledegg*” and “*pancake*”, both involving cooking with a pan.

4.5.5.2 Temporal Coherence

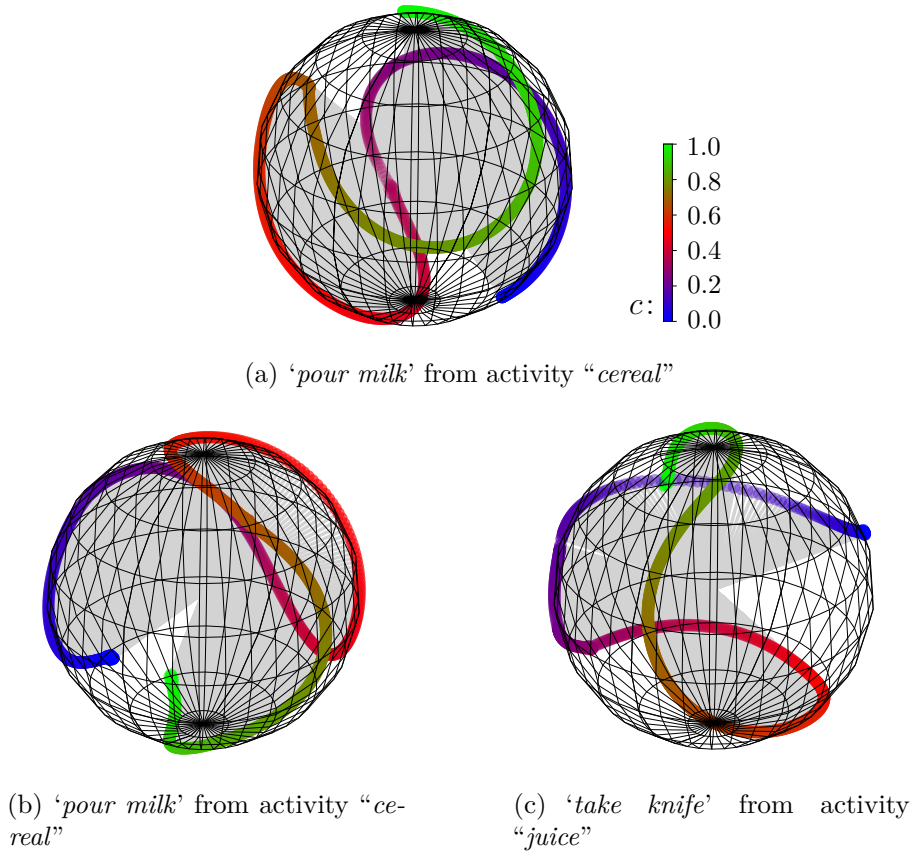


Figure 4.3: T-SNE visualizations of generated action segments.

To assess the temporal coherence of the generated samples, we employ T-SNE [43] to visualize action segments generated by our TCA model. Every point in the sphere corresponds to a frame feature. Each segment comprises 1,000 frames, each sharing a common action label a and latent variable z , with an evenly strided c .

Specifically, we employ the TCA model trained on “*cereal*” data to generate an action segment of ‘*pour milk*’ in Fig. 4.3(a). Fig. 4.3(b) depicts the same action

of ‘*pour milk*’ for “*cereal*” but is generated using a different latent variable z , highlighting the diversity in trajectories. Fig. 4.3(c) shows a generated segment of a different action ‘*take knife*’ from “*juice*”. All visualized action segments exhibit continuous properties in the feature space, corresponding to the gradual increase in the coherence variable c , showcasing the temporal coherence between the frame features.

Chapter 5

Conclusion

This thesis introduced the incremental temporal action segmentation task in an endeavor to push TAS research into developing approaches for models with continual learning capabilities. We proposed a temporally coherent action model for incremental procedural video understanding from the perspective of video data replay. Departing from the conventional frame-exemplar data replay prevalent in action recognition, our approach adopts a generative model. In addition, we facilitate the modeling of temporal coherence in the actions by introducing a conditioning variable to the generative model. Our design guarantees both feature diversity and temporal coherence in the replay data, resulting in a significant performance improvement on three action segmentation benchmarks compared to the baselines.

5.1 Limitations

This work presupposes the presence of dense labels for all videos within each incremental task, and our TCA model relies on such dense labels for effective learning. However, such an annotation process can be expensive in a real-world scenario. Additionally, while we can demonstrate the temporal coherence of the features generated by our TCA model, a qualitative evaluation of the features remains challenging because their existence in a feature space makes them difficult to decode into a human-readable format.

5.2 Future Work

Much research into weaker forms of supervision in TAS has been conducted, given the significant costs involved in curating a fully temporally annotated dataset.

Weaker forms of supervision such as action transcripts [30] and timestamp supervision [39, 49] have achieved great results in recent years. Exploring the iTAS setup with weak supervision would be a promising direction for future research.

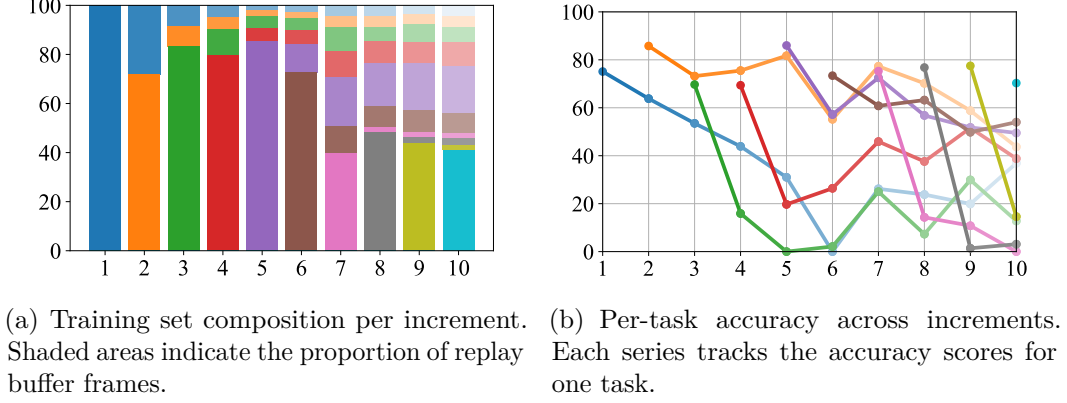


Figure 5.1: Effect of task sequence order to training set composition and model performance.

As was observed in Section 4.5.4, model performance can be affected by task sequence order. To visualize the effect of task ordering on the training set and model performance, we plot the training set composition and per-task accuracy scores in Fig. 5.1(a) and Fig. 5.1(b). Results shown are from train-test split 1 of Breakfast using task sequence order from seed 42.

We hypothesize that due to the frame imbalance between tasks, tasks with longer videos (e.g. task 5) can dominate the training set and negatively affect model performance in other tasks, especially if encountered early in the sequence. A more considered approach to action sequence sampling or replay memory management can be explored in future work to alleviate this phenomenon.

Temporal action segmentation is an exciting and rapidly advancing field. We must progress towards developing models with incremental learning capabilities to reach adoption into real-world systems. We hope our work will spur others in the field to consider continual learning when designing algorithms for TAS.

Bibliography

- [1] J.-B. Alayrac, P. Bojanowski, N. Agrawal, J. Sivic, I. Laptev, and S. Lacoste-Julien, “Unsupervised learning from narrated instruction videos”, in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4575–4583.
- [2] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, “Memory aware synapses: Learning what (not) to forget”, in *Computer Vision – ECCV 2018*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds., Cham: Springer International Publishing, 2018, pp. 144–161.
- [3] L. Alssum, J. León Alcázar, M. Ramazanov, C. Zhao, and B. Ghanem, “Just a glimpse: Rethinking temporal information for video continual learning”, in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, pp. 2474–2483.
- [4] J. Bang, H. Kim, Y. Yoo, J.-W. Ha, and J. Choi, “Rainbow memory: Continual learning with a memory of diverse samples”, in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 8214–8223.
- [5] J. Bang, H. Koh, S. Park, H. Song, J.-W. Ha, and J. Choi, “Online continual learning on a contaminated data stream with blurry task boundaries”, in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 9265–9274.
- [6] F. Becattini, T. Uricchio, L. Seidenari, L. Ballan, and A. D. Bimbo, “Am i done? predicting action progress in videos”, *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 16, no. 4, Dec. 2020.
- [7] J. Carreira and A. Zisserman, “Quo vadis, action recognition? a new model and the kinetics dataset”, in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4724–4733.

BIBLIOGRAPHY

- [8] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars, “A continual learning survey: Defying forgetting in classification tasks”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3366–3385, 2022.
- [9] G. Ding, F. Sener, and A. Yao, “Temporal action segmentation: An analysis of modern techniques”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 2, pp. 1011–1030, 2024.
- [10] G. Ding and A. Yao, “Leveraging action affinity and continuity for semi-supervised temporal action segmentation”, in *Computer Vision – ECCV 2022*, S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, Eds., Cham: Springer Nature Switzerland, 2022, pp. 17–32.
- [11] Y. A. Farha and J. Gall, “Ms-tcn: Multi-stage temporal convolutional network for action segmentation”, in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3570–3579.
- [12] A. Fathi, X. Ren, and J. M. Rehg, “Learning to recognize objects in egocentric activities”, in *CVPR 2011*, 2011, pp. 3281–3288.
- [13] R. M. French, “Catastrophic forgetting in connectionist networks”, *Trends in Cognitive Sciences*, vol. 3, no. 4, pp. 128–135, 1999.
- [14] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets”, in *Advances in Neural Information Processing Systems*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, Eds., vol. 27, Curran Associates, Inc., 2014.
- [15] T. L. Hayes, K. Kafle, R. Shrestha, M. Acharya, and C. Kanan, “Remind your neural network to prevent catastrophic forgetting”, in *Computer Vision – ECCV 2020*, A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, Eds., Cham: Springer International Publishing, 2020, pp. 466–483.
- [16] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition”, in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

BIBLIOGRAPHY

- [17] S. Hershey, S. Chaudhuri, D. P. W. Ellis, J. F. Gemmeke, A. Jansen, R. C. Moore, M. Plakal, D. Platt, R. A. Saurous, B. Seybold, M. Slaney, R. J. Weiss, and K. Wilson, “Cnn architectures for large-scale audio classification”, in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 131–135.
- [18] S. Hou, X. Pan, C. C. Loy, Z. Wang, and D. Lin, “Learning a unified classifier incrementally via rebalancing”, in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 831–839.
- [19] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift”, in *Proceedings of the 32nd International Conference on Machine Learning*, F. Bach and D. Blei, Eds., ser. Proceedings of Machine Learning Research, vol. 37, Lille, France: PMLR, Jul. 2015, pp. 448–456.
- [20] A. Iscen, J. Zhang, S. Lazebnik, and C. Schmid, “Memory-efficient incremental learning through feature adaptation”, in *Computer Vision – ECCV 2020*, A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, Eds., Cham: Springer International Publishing, 2020, pp. 699–715.
- [21] K. Ishihara, G. Nakano, and T. Inoshita, “Mcfm: Mutual cross fusion module for intermediate fusion-based action segmentation”, in *2022 IEEE International Conference on Image Processing (ICIP)*, 2022, pp. 1701–1705.
- [22] H. Jung, J. Ju, M. Jung, and J. Kim, “Less-forgetful learning for domain expansion in deep neural networks”, *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018.
- [23] W. Kay, J. Carreira, K. Simonyan, B. Zhang, C. Hillier, S. Vijayanarasimhan, F. Viola, T. Green, T. Back, P. Natsev, M. Suleyman, and A. Zisserman, “The kinetics human action video dataset”, 2017. arXiv: **1705.06950** [cs.CV].
- [24] D. P. Kingma and M. Welling, “Auto-encoding variational bayes”, in *2nd International Conference on Learning Representations (ICLR)*, 2014.
- [25] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, “Overcoming catastrophic forgetting

BIBLIOGRAPHY

- in neural networks”, *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [26] H. Koh, D. Kim, J.-W. Ha, and J. Choi, “Online continual learning on class incremental blurry task configuration with anytime inference”, in *10th International Conference on Learning Representations (ICLR)*, 2022.
 - [27] Y. Kong and Y. Fu, “Human Action Recognition and Prediction: A Survey”, *International Journal of Computer Vision*, vol. 130, no. 5, pp. 1366–1401, May 2022.
 - [28] H. Kuehne, A. Arslan, and T. Serre, “The language of actions: Recovering the syntax and semantics of goal-directed human activities”, in *2014 IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 780–787.
 - [29] H. Kuehne, J. Gall, and T. Serre, “An end-to-end generative framework for video segmentation and recognition”, in *2016 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2016, pp. 1–8.
 - [30] H. Kuehne, A. Richard, and J. Gall, “Weakly supervised learning of actions from transcripts”, *Computer Vision and Image Understanding*, vol. 163, pp. 78–89, 2017, Language in Vision.
 - [31] A. Kukleva, H. Kuehne, F. Sener, and J. Gall, “Unsupervised learning of action classes with continuous temporal embedding”, in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 12 058–12 066.
 - [32] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, “Temporal convolutional networks for action segmentation and detection”, in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1003–1012.
 - [33] C. Lea, R. Vidal, and G. D. Hager, “Learning convolutional action primitives for fine-grained action recognition”, in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 1642–1649.
 - [34] T. Lesort, H. Caselles-Dupré, M. Garcia-Ortiz, A. Stoian, and D. Filliat, “Generative models from the perspective of continual learning”, in *2019 International Joint Conference on Neural Networks (IJCNN)*, 2019, pp. 1–8.

BIBLIOGRAPHY

- [35] J. Li, P. Lei, and S. Todorovic, “Weakly supervised energy-based learning for action segmentation”, in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 6242–6250.
- [36] M. Li, L. Chen, Y. Duarr, Z. Hu, J. Feng, J. Zhou, and J. Lu, “Bridge-prompt: Towards ordinal action understanding in instructional videos”, in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 19 848–19 857.
- [37] S. Li, Y. A. Farha, Y. Liu, M.-M. Cheng, and J. Gall, “Ms-tcn++: Multi-stage temporal convolutional network for action segmentation”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 6, pp. 6647–6658, 2023.
- [38] Y. Li, Y. Li, and N. Vasconcelos, “Resound: Towards action recognition without representation bias”, in *Computer Vision – ECCV 2018*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds., Cham: Springer International Publishing, 2018, pp. 520–535.
- [39] Z. Li, Y. A. Farha, and J. Gall, “Temporal action segmentation from timestamp supervision”, in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 8361–8370.
- [40] Z. Li and D. Hoiem, “Learning without forgetting”, in *Computer Vision – ECCV 2016*, B. Leibe, J. Matas, N. Sebe, and M. Welling, Eds., Cham: Springer International Publishing, 2016, pp. 614–629.
- [41] J. Lin, C. Gan, and S. Han, “Tsm: Temporal shift module for efficient video understanding”, in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 7082–7092.
- [42] D. Liu, Q. Li, A.-D. Dinh, T. Jiang, M. Shah, and C. Xu, “Diffusion action segmentation”, in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 10 105–10 115.
- [43] L. van der Maaten and G. Hinton, “Visualizing data using t-sne”, *Journal of Machine Learning Research*, vol. 9, no. 86, pp. 2579–2605, 2008.

BIBLIOGRAPHY

- [44] M. Masana, X. Liu, B. Twardowski, M. Menta, A. D. Bagdanov, and J. van de Weijer, “Class-incremental learning: Survey and performance evaluation on image classification”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 5, pp. 5513–5533, 2023.
- [45] B. Ni, X. Yang, and S. Gao, “Progressively parsing interactional objects for fine grained action detection”, in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1020–1028.
- [46] J. Park, M. Kang, and B. Han, “Class-incremental learning for action recognition in videos”, in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 13 678–13 687.
- [47] Y. Pei, Z. Qing, J. CEN, X. Wang, S. Zhang, Y. Wang, M. Tang, N. Sang, and X. Qian, “Learning a condensed frame for memory-efficient video class-incremental learning”, in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35, Curran Associates, Inc., 2022, pp. 31 002–31 016.
- [48] F. Perronnin, J. Sánchez, and T. Mensink, “Improving the fisher kernel for large-scale image classification”, in *Computer Vision – ECCV 2010*, K. Daniilidis, P. Maragos, and N. Paragios, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 143–156.
- [49] R. Rahaman, D. Singhanian, A. Thiery, and A. Yao, “A generalized and robust framework for timestamp supervision in temporal action segmentation”, in *Computer Vision – ECCV 2022*, S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, Eds., Cham: Springer Nature Switzerland, 2022, pp. 279–296.
- [50] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, “Icarl: Incremental classifier and representation learning”, in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 5533–5542.
- [51] A. Richard and J. Gall, “Temporal action detection using a statistical language model”, in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 3131–3140.

BIBLIOGRAPHY

- [52] A. Richard, H. Kuehne, and J. Gall, “Weakly supervised action learning with rnn based fine-to-coarse modeling”, in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1273–1282.
- [53] A. Richard, H. Kuehne, and J. Gall, “Action sets: Weakly supervised action segmentation without ordering constraints”, in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 5987–5996.
- [54] A. Richard, H. Kuehne, A. Iqbal, and J. Gall, “Neuralnetwork-viterbi: A framework for weakly supervised video learning”, in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 7386–7395.
- [55] M. Rohrbach, S. Amin, M. Andriluka, and B. Schiele, “A database for fine grained activity detection of cooking activities”, in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, 2012, pp. 1194–1201.
- [56] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, “ImageNet Large Scale Visual Recognition Challenge”, *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, Dec. 2015, ISSN: 1573-1405.
- [57] F. Sener and A. Yao, “Unsupervised learning and segmentation of complex activities from video”, in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8368–8376.
- [58] H. Shin, J. K. Lee, J. Kim, and J. Kim, “Continual learning with deep generative replay”, in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30, Curran Associates, Inc., 2017.
- [59] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition”, *3rd International Conference on Learning Representations (ICLR)*, 2015.
- [60] B. Singh, T. K. Marks, M. Jones, O. Tuzel, and M. Shao, “A multi-stream bi-directional recurrent neural network for fine-grained action detection”, in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1961–1970.

BIBLIOGRAPHY

- [61] D. Singhania, R. Rahaman, and A. Yao, “C2f-tcn: A framework for semi- and fully-supervised temporal action segmentation”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 10, pp. 11 484–11 501, 2023.
- [62] K. Sohn, H. Lee, and X. Yan, “Learning structured output representation using deep conditional generative models”, in *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28, Curran Associates, Inc., 2015.
- [63] K. Soomro, A. R. Zamir, and M. Shah, “Ucf101: A dataset of 101 human actions classes from videos in the wild”, *arXiv preprint arXiv:1212.0402*, 2012.
- [64] Y. Souri, Y. A. Farha, E. Bahrami, G. Francesca, and J. Gall, “Robust action segmentation from timestamp supervision”, in *33rd British Machine Vision Conference 2022, BMVC 2022, London, UK, November 21-24, 2022*, BMVA Press, 2022.
- [65] S. Stein and S. J. McKenna, “Combining embedded accelerometers with computer vision for recognizing food preparation activities”, in *Proceedings of the 2013 ACM International Joint Conference on Pervasive and Ubiquitous Computing*, ser. UbiComp ’13, Zurich, Switzerland: Association for Computing Machinery, 2013, pp. 729–738.
- [66] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. u. Kaiser, and I. Polosukhin, “Attention is All you Need”, in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30, Curran Associates, Inc., 2017.
- [67] G. M. van de Ven, T. Tuytelaars, and A. S. Tolias, “Three types of incremental learning”, *Nature Machine Intelligence*, vol. 4, no. 12, pp. 1185–1197, Dec. 2022.
- [68] A. Villa, K. Alhamoud, V. Escorcia, F. C. Heilbron, J. L. Alcázar, and B. Ghanem, “Vclimb: A novel video class incremental learning benchmark”, in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 19 013–19 022.

BIBLIOGRAPHY

- [69] H. Wang, A. Kläser, C. Schmid, and C.-L. Liu, “Action recognition by dense trajectories”, in *CVPR 2011*, 2011, pp. 3169–3176.
- [70] H. Wang and C. Schmid, “Action recognition with improved trajectories”, in *2013 IEEE International Conference on Computer Vision*, 2013, pp. 3551–3558.
- [71] L. Wang, Y. Xiong, Z. Wang, Y. Qiao, D. Lin, X. Tang, and L. Van Gool, “Temporal segment networks: Towards good practices for deep action recognition”, in *Computer Vision – ECCV 2016*, B. Leibe, J. Matas, N. Sebe, and M. Welling, Eds., Cham: Springer International Publishing, 2016, pp. 20–36.
- [72] Y. Wu, Y. Chen, L. Wang, Y. Ye, Z. Liu, Y. Guo, and Y. Fu, “Large scale incremental learning”, in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 374–382.
- [73] Y. Xiang, Y. Fu, P. Ji, and H. Huang, “Incremental learning using conditional adversarial networks”, in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 6618–6627.
- [74] F. Yi, H. Wen, and T. Jiang, “Asformer: Transformer for action segmentation”, in *The British Machine Vision Conference (BMVC)*, 2021.
- [75] F. Zenke, B. Poole, and S. Ganguli, “Continual learning through synaptic intelligence”, in *Proceedings of the 34th International Conference on Machine Learning*, D. Precup and Y. W. Teh, Eds., ser. Proceedings of Machine Learning Research, vol. 70, PMLR, Jun. 2017, pp. 3987–3995.
- [76] J. Zhang, J. Zhang, S. Ghosh, D. Li, S. Tasci, L. Heck, H. Zhang, and C.-C. Jay Kuo, “Class-incremental learning via deep model consolidation”, in *2020 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1120–1129.
- [77] D. Zhukov, J.-B. Alayrac, R. G. Cinbis, D. Fouhey, I. Laptev, and J. Sivic, “Cross-task weakly supervised learning from instructional videos”, in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3532–3540.